

ELEC/COEN 390

Engineering team design project

MD NAZMUL HOQ

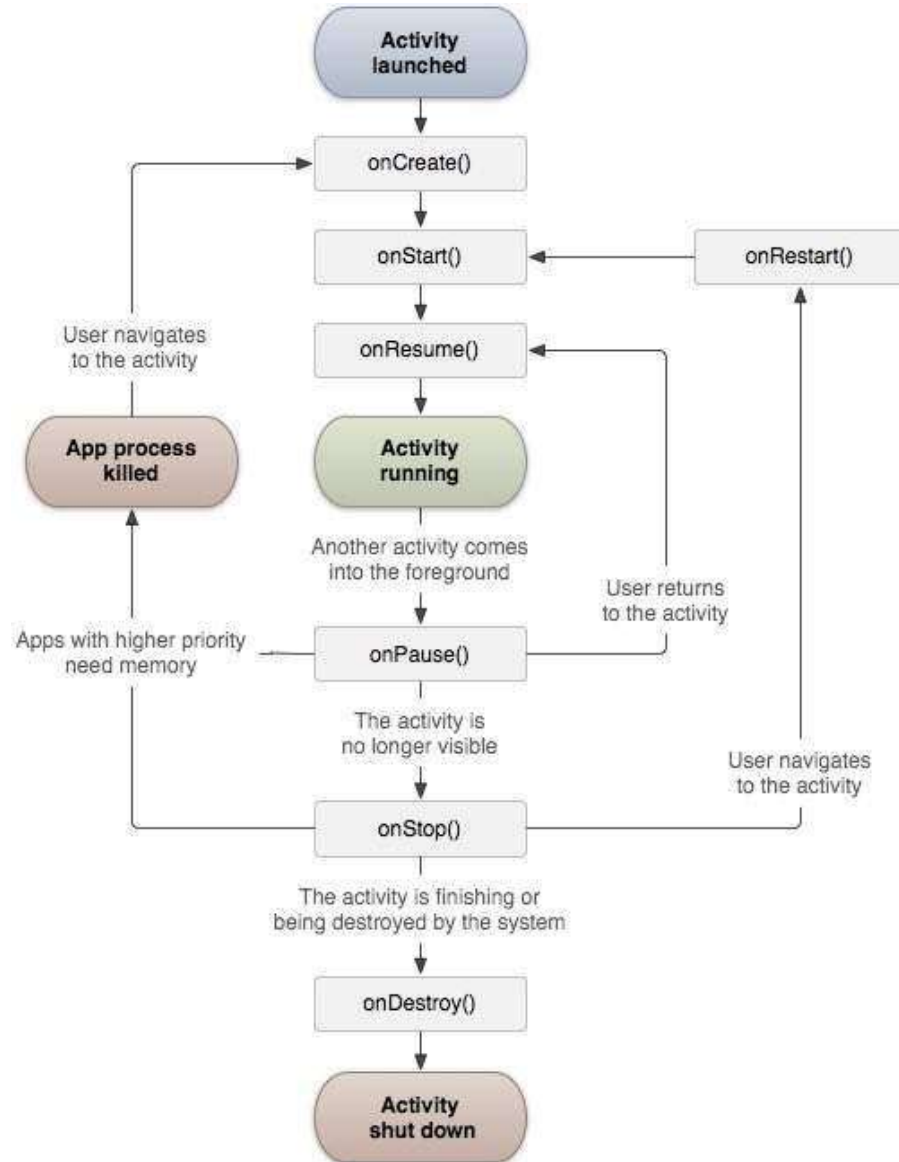
KHADIZA SARWAR MOURY

CONCORDIA UNIVERSITY

Outline

2

- ▶ Review the Activity's Life Cycle
- ▶ Shared Preferences
- ▶ Demo
- ▶ Basic Software Design Principles
- ▶ MVC Design Pattern
- ▶ Some Naming Conventions and Tips for Clean Code

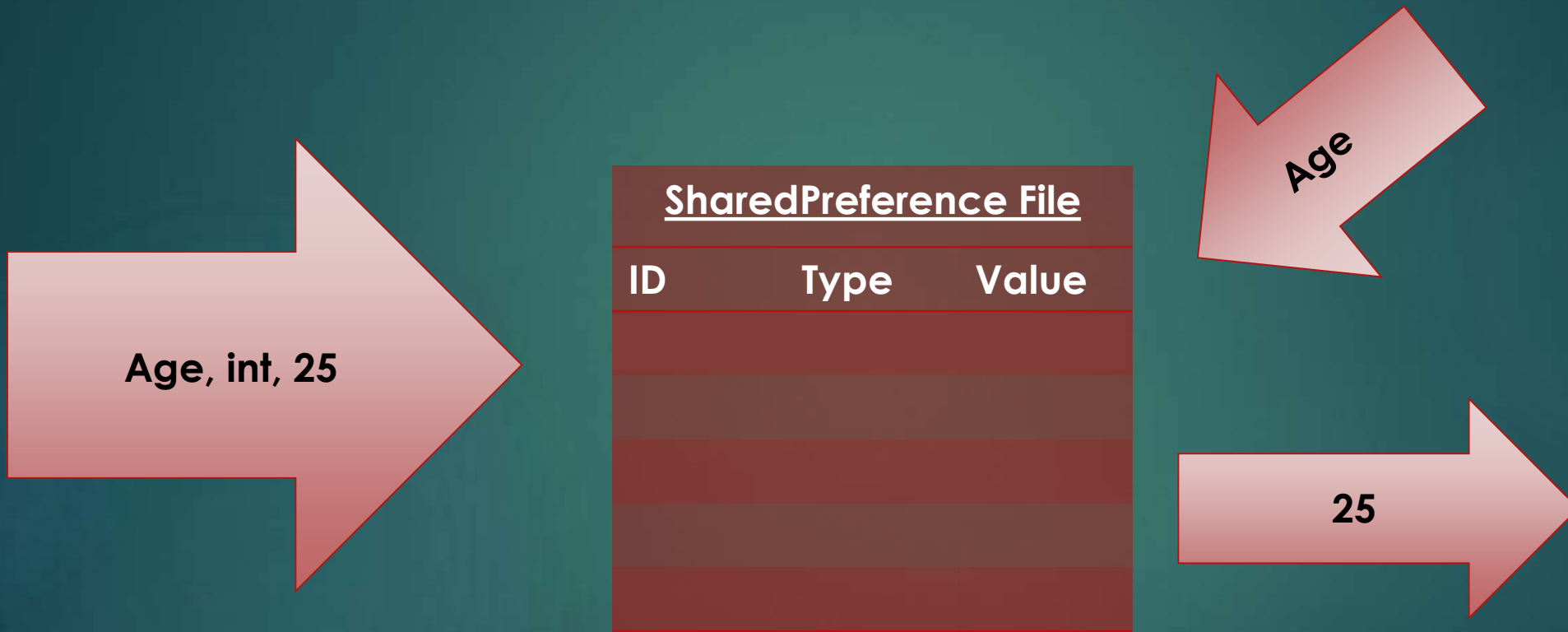


Activity's Life Cycle

Shared Preferences

- ▶ A SharedPreferences object points to a file containing key-value pairs and provides simple methods to read and write them.
- ▶ Each SharedPreferences file is managed by the framework and can be private or shared.
- ▶ use [`getSharedPreferences\(\)`](#) if you need multiple shared preference files identified by name, which you specify with the first parameter.
- ▶ use [`getPreferences\(\)`](#) if you need to use only one shared preference file for the activity. Because this retrieves a default shared preference file that belongs to the activity, you don't need to supply a name.
- ▶ Source: <https://developer.android.com/training/basics/data-storage/shared-preferences.html#ReadSharedPreference>

5



Shared Preferences

To write to a shared preference use an Editor

```
SharedPreferences sharedPref =  
getActivity().getPreferences(Context.MODE_PRIVATE);  
SharedPreferences.Editor editor = sharedPref.edit();  
editor.putInt(getString(R.string.saved_high_score), newHighScore);  
editor.commit();
```

Shared Preferences

To read from a shared Preference:

```
SharedPreferences sharedPref = getActivity().getPreferences(Context.MODE_PRIVATE);  
int defaultValue = getResources().getInteger(R.string.saved_high_score_default);  
int highScore = sharedPref.getInt(getString(R.string.saved_high_score), defaultValue);
```

Demo

Basic Software Design Principles

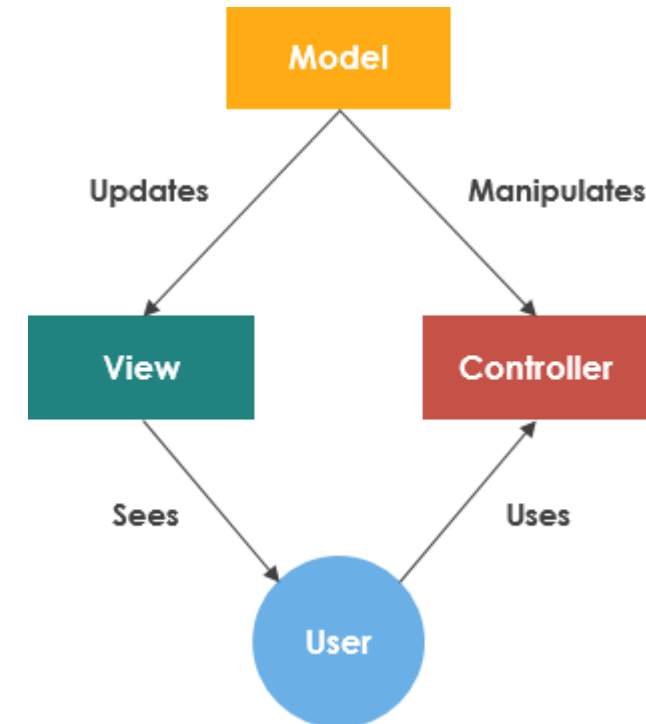
9

- ▶ **Separation of Concern (SoC)** is a design principle for separating a computer program into distinct sections, such that each section addresses a separate concern.
- ▶ **Model-View-Controller (MVC)** design pattern is an excellent example of separating these concerns

What is MVC

10

- ▶ M : Model
 - ▶ Database, files, etc.
- ▶ V : View
 - ▶ Graphs, stats, charts, etc.
- ▶ C : Controller
 - ▶ Buttons, text fields, etc.
- ▶ Never repeat yourself
- ▶ Structured Programming
- ▶ Cleaner Design



Clean Code

11

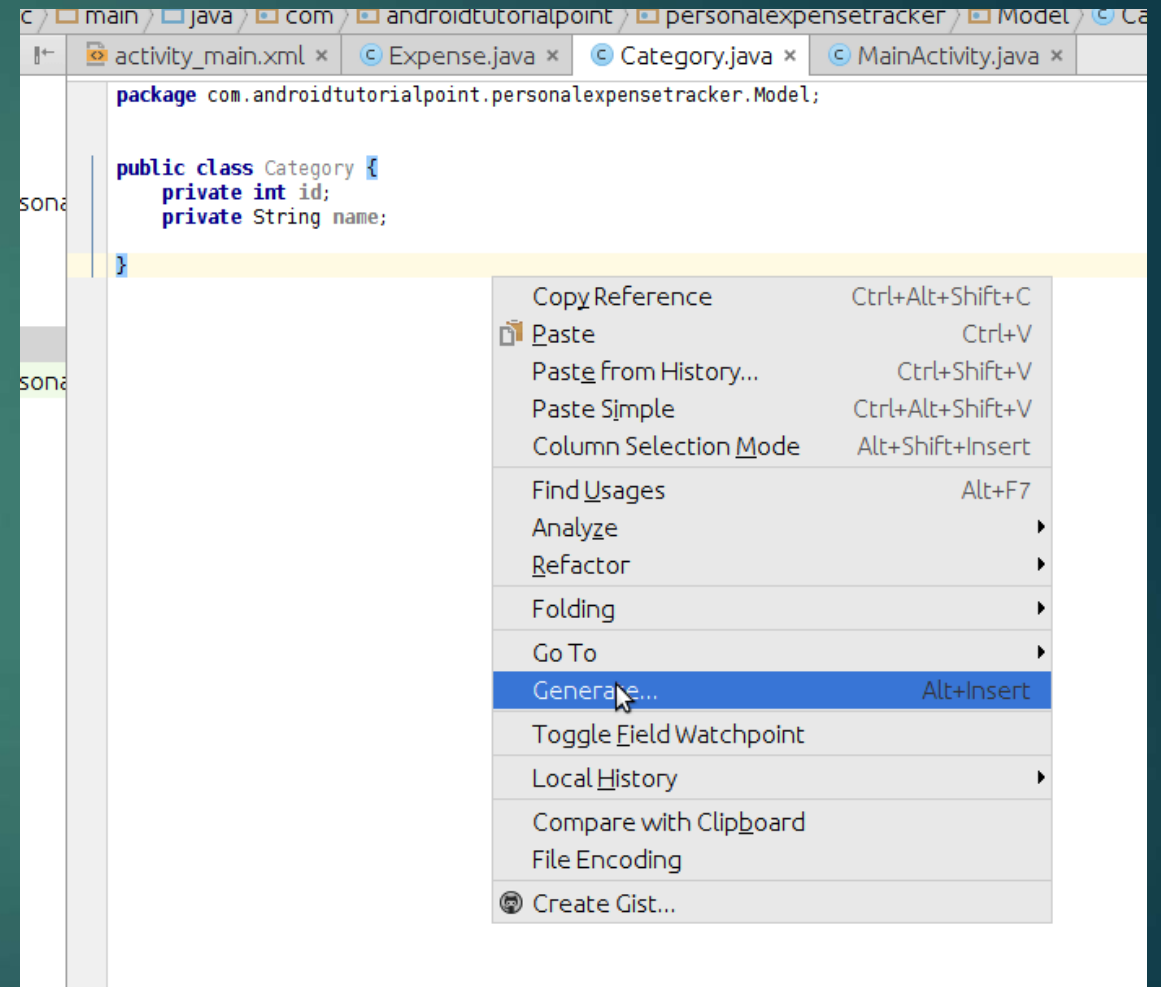
- ▶ USE CODE -> **AUTO-INDENT LINES !**
- ▶ Comment Judiciously!
 - ▶ At the beginning of every java file (class) a comment should describe what this class is doing
 - ▶ Before every method, comment what the method is doing
 - ▶ Use //TODO: to identify missing features or ideas for future code
- ▶ Write Short Methods
- ▶ Define Fields in Standard Places (at the top of the file)
- ▶ Follow Field Naming Conventions
 - ▶ Non-public, non-static field names start with m.
 - ▶ Static field names start with s.
 - ▶ Other fields start with a lower case letter.
 - ▶ Public static final fields (constants) are ALL_CAPS_WITH_UNDERSCORES.
- ▶ Don't leave excessive empty lines

(TRY TO)
ALWAYS BE
CONSISTENT

Android Studio Helps!

13

- ▶ Android Studio helps you follow good code guidelines!
- ▶ It can do a lot of things for you:
 - ▶ Generate Getters/Setters
 - ▶ Create “Singleton” classes
 - ▶ Make Android specific classes
 - ▶ Track “TODOs”
 - ▶ Check Variables (limited)
 - ▶ Warn about code duplication (optional)



Team Projects

14

Prepare

- Make sure you are confident with the project you choose!

Minutes

- Keep a reference of your meetings

Team Work

- Follow the decisions the team makes

Time Management

- Start early!

Documentation

- Document everything very well!

How to help yourself?

15

- ▶ Use IDE autocompletion and code generation
- ▶ Compile your code often
- ▶ Change only one feature at a time
- ▶ Fix errors based on the error messages and IDE hints
 - ▶ Compile errors & runtime errors
- ▶ Use debugging & break points to step through the code
- ▶ Use logging statements to create informative logs
- ▶ Use try catch statements to handle errors elegantly
- ▶ Refer to documentation and tutorials...

How to use references?

16

- ▶ Make sure the reference is current
 - ▶ 3-5 years old is sometimes already obsolete
- ▶ Make sure the reference has authority
- ▶ Make sure the coding strategy is good
 - ▶ Not just “functional”
- ▶ Use the reference to understand the principles and write your own version to accomplish your specific goal

Note about using references

17

- ▶ References should be used to understand which elements of the code are important for a given functionality
- ▶ The important elements may be Classes, Functions, Data Types, Algorithms, or else.
- ▶ Knowing these elements will then allow you to write your own code using these specific elements.
- ▶ Essentially, this is the workflow we recommend:
 - ▶ Read the reference, take quick notes about the important elements
 - ▶ Close/minimize the reference
 - ▶ Work on your code to implement the desired features.
- ▶ Using this workflow will help you better memorize the key elements used to implement any given functionality

A few don'ts...

18

- ▶ **DO NOT copy/paste code from any source** other than your own project (even that is bad practice, better to practice your typing).
- ▶ DO NOT start a project using an existing project **if your goal is not explicitly stated** to be “extending this existing Open-Source project”.
- ▶ DO NOT reuse whole source code files unless they are libraries and **explicitly cited** as the work of the original authors.
- ▶ **DO NOT use a past solution** to an ELEC/COEN390 assignment as a reference. Consider that most have TERRIBLE coding quality...
- ▶ **DO NOT make your solution public** before the semester has ended.

Questions?